

Special Participation B

MAML

Me:

Question

Considering the plot of regression test loss versus `n_train_post`, **how does the performance of the meta-learned feature weights compare to the case where all feature weights are set to 1?** Additionally, **how does their performance compare to the oracle**, which performs regression using only the features present in the data? Can you **explain the reason for the downward spike observed at `n_train_post = 32`**? Include the answer in your written submission of the written assignment.

Question

By examining the changes in feature weights over time during meta-learning, **can you justify the observed improvement in performance?** Specifically, can you **explain why certain feature weights are driven towards zero?** Include the answer in your written submission of the written assignment.

Included context for notebook

Gemini Pro

Based on the code and experimental setup in [q_maml.ipynb](#), here are the answers to your questions:

Question 1

Comparison of Meta-learned Feature Weights vs. All Feature Weights Set to 1:

The meta-learned feature weights result in significantly **lower regression test loss** compared to the case where all feature weights are set to 1 (the "Init" curve).

- With all weights set to 1, the model treats all 501 features as equally important. Since the number of features ($d=501$) is much larger than the number of training points (e.g., `n_train_post` around 10-100), the standard least squares/ridge regression overfits massively to the noise and the irrelevant features, leading to high test error.
- The meta-learned model has learned to down-weight irrelevant features, effectively reducing the dimensionality of the problem, which leads to much better generalization and lower test loss.

Comparison to the Oracle:

The performance of the meta-learned feature weights **closely approaches the performance of the Oracle**.

- The Oracle knows exactly which frequencies are present in the data (`k_idx`) and uses only those features (weights=1 for `k_idx`, 0 otherwise).
- The meta-learner successfully discovers this structure during training, learning to assign high weights to the true features and near-zero weights to the others. As a result, its test loss curve tracks the Oracle's curve very closely.

Reason for the Downward Spike at `n_train_post = 32`:

The downward spike (drop in test loss) at `n_train_post = 32` occurs because **32 is exactly the number of training points used in the inner loop (`n_train_inner = 32`) during meta-training**.

- Since the training data is generated on a deterministic **grid** (`x_type = 'grid'`), the aliasing patterns and properties of the data are identical across all tasks when $N=32$.
- The meta-learner optimizes the feature weights to minimize the loss specifically for this 32-point grid configuration. It learns a prior that is "overfitted" or perfectly specialized to the specific aliasing and structure of a 32-point grid.
- When tested with `n_train_post = 32`, the test condition perfectly matches the meta-training condition, resulting in exceptionally low loss. At other sample sizes, the grid spacing (and thus aliasing) differs, so the specialized weights are slightly less effective, causing the loss to be higher than at the "resonant" point of 32.

Question 2

Justification of Improvement in Performance:

The observed improvement in performance is justified by the model **learning the sparsity structure of the task**.

- The underlying ground truth functions are generated using only a small subset of Fourier features (indices 5 to 10).
- Over time, the meta-learning process adjusts the feature weights to reflect this reality. By increasing the weights of the relevant features and decreasing the weights of irrelevant ones, the model transforms the ill-posed, high-dimensional regression problem (501 features) into a well-posed, low-dimensional one (approx. 6 features). This drastically reduces the variance of the estimator and improves generalization.

Why Certain Feature Weights are Driven Towards Zero:

Certain feature weights are driven towards zero because **they correspond to features that are irrelevant to the tasks (noise features)**.

- Features not in `k_idx` (and not aliased to them) have no correlation with the target labels `yy`.
- If these features have non-zero weights, the regression algorithm will use them to fit the noise in the training data (overfitting), which increases the test error.
- To minimize the meta-loss (the test error across tasks), the gradient descent

updates drive the weights of these irrelevant features to zero. This effectively performs **soft feature selection**, removing the "nuisance" dimensions that only contribute to variance without reducing bias.

Me:

Question

With `num_gd_steps = 5`, does meta-learning contribute to improved performance during test time? Furthermore, if we change this to `num_gd_steps = 1`, does meta-learning continue to function effectively? Include the answer in your written submission of the written assignment.

Gemini Pro: Was not able to generate an output. It could not read the images generated by the previous cell

Me:

MAML for Classification

Suppose we want to learn an effective set of feature weights for performing classification tasks. We will use the same setup as the regression problem, but now our training data consists of pairs (x_i, z_i) , where $z_i = \text{sgn}(f(x_i))$. The function f represents the underlying true function that is sampled from a distribution D_T as before.

Given a set of feature weights and their corresponding weighted features ϕ_w , we solve the logistic regression problem to learn a set of coefficients α . Using these coefficients, we assign a label to a test point x_{test} as $\hat{z}_{\text{test}} = \text{sgn}(\langle \alpha, \phi_w(x_{\text{test}}) \rangle)$. The test loss of interest is the classification error $\mathbb{E}[\hat{z}_{\text{test}} \neq z_{\text{test}}]$, where the expectation is taken over the randomness in the test point. Since the test loss is not differentiable, we use the logistic loss during training (logistic regression).

Next, we describe the process of learning the coefficients $\alpha \in \mathbb{R}^d$. We have training data pairs (x_i, z_i) , where x_i represents a data point, and z_i denotes its label from the set $+1, -1$. For each point x_i , we have a set of weighted features $\phi_w(x_i)$.

Recall that a standard logistic function is defined as

$$g(t) = \frac{1}{e^{-t} + 1}$$

(refer to: https://en.wikipedia.org/wiki/Logistic_function), which possesses the useful property of $g(-t) = 1 - g(t)$. Consequently, we can use it to model the binary probability: given a value t that might belong to two classes with labels $+1$ and -1 , we can model the probability of t being part of class $+1$ as $p_1(t) = g(t)$, and conveniently obtain $1 - g(t) = p_{-1}(t)$. In our classification problem setup, this $t = z_i \langle \alpha, \phi_w(x_i) \rangle$. Thus, $g(t)$ can be interpreted as the probability of "correct classification", because for a correct prediction, $t = z_i \langle \alpha, \phi_w(x_i) \rangle$ will always be positive, and its probability $g(t)$ will be appropriately limited to 1.

Therefore, to maximize the total log probability of predicting the correct label on all datapoints we find the α that maximizes

$$\sum_i \log \frac{1}{e^{-z_i \langle \alpha, \phi_w(x_i) \rangle} + 1}.$$

Flipping the sign, this is equivalent to finding the α that minimizes the loss

$$-\sum_i \log \frac{1}{e^{-z_i \langle \alpha, \phi_w(x_i) \rangle} + 1},$$

as listed here: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

One way to find this α is by using gradient descent similar to what we did in the regression setting. In the regression setting we used the squared loss but in the classification setting we use the logistic loss.

Gemini Pro:

Additions:

```
z_meta_t = my_sign(y_meta_t)
loss = torch.sum(torch.nn.Softplus()(-z_meta_t * y_meta_pred_t))
```

```
for j in range(num_gd_steps):
    y_pred = mod(features_t)

#####
    # TODO: Complete the following code

#####
    # 1. Get the binary labels z (+1 or -1)
    # We use y_t which is the tensor version of
y available in this scope
    z_t = my_sign(y_t)

    # 2. Compute the logistic loss: mean(log(1 +
exp(-z * prediction)))
    # We use mean as per the regression hint
(loss = torch.mean(...))
    loss = torch.mean(torch.log(1 + torch.exp(-
z_t * y_pred)))

#####
```

```

opt_inner.zero_grad()
loss.backward(retain_graph=True)
opt.step(loss)

```

I have completed the code for MAML classification in the notebook [q_maml.ipynb](#).

I implemented the logistic loss function as requested, using `torch.nn.Softplus` for numerical stability, which computes $\log(1+\exp(x))\log(1+\exp(x))$.

Changes made:

1. `meta_update_classification`:

- Calculated the binary labels `zmetazmeta` using `my_sign(y_meta_t)`.
- Computed the total logistic loss using `sum` reduction:
 -
 -
 -
 -

2. `meta_learning_classification` (Inner Loop):

- Calculated the binary labels `ztzt` using `my_sign(y_t)`.
- Computed the mean logistic loss for the inner gradient descent steps:
 -
 -
 -
 -

`.Softplus()(-z_t * y_pred))`

These changes align with the provided mathematical formulation and the structure of the regression implementation in the notebook. These changes align with the provided mathematical formulation and the structure of the regression implementation in the notebook.

Me:

Question

Based on the plot of classification error versus `n_train_post`, how does the performance of the meta-learned feature weights compare to the case where all feature weights are 1? How does the performance of the meta-learned feature weights compare to the oracle (which performs logistic regression using only the features present in the data)? Include the answer in your written submission of the written assignment.

Question

By observing the evolution of the feature weights over time as we perform meta-learning, can you justify the improvement in performance? Specifically, can you explain why some feature weights are being driven towards zero? Include the answer in your written submission of the written assignment.

Gemini pro:

Failed to generate because of the pictures

Me: Copied the questions and information into gemini pro and asked it to solve it

Gemini pro:

Based on the provided plots and the context of the MAML (Model-Agnostic Meta-Learning) assignment, here are the answers to your questions.

Performance Comparison

How does the performance of the meta-learned feature weights compare to the case where all feature weights are 1?

The performance of the meta-learned feature weights (Blue line, "Iteration: 99") is significantly better than the initialization case where all feature weights are 1 (Orange line, "Init").

- **Observation:** In the top-left plot ("Test classification error"), the Blue line starts at a much lower error rate and descends rapidly even with very few training points (`n_train_post`). The Orange line starts with high error and requires significantly more data points to achieve a comparable error rate.
- **Interpretation:** The initialization treats all features equally (both relevant features and noise). The meta-learned model has successfully learned a "prior"

that emphasizes useful features. This allows the model to learn the specific task much more efficiently (few-shot learning), effectively filtering out high-dimensional noise right from the start.

How does the performance of the meta-learned feature weights compare to the oracle?

The performance of the **meta-learned feature weights tracks the Oracle (Green line) very closely**, almost matching it.

- **Observation:** The Blue line is consistently near the Green line, often overlapping or staying within the error margins.
 - **Interpretation:** The Oracle represents the theoretical lower bound for error because it "knows" exactly which features are relevant and ignores the noise completely. The fact that the meta-learned model performs similarly to the Oracle indicates that the meta-learning process has successfully identified the underlying support (the subset of informative features) and is effectively ignoring the irrelevant dimensions.
-

Feature Weights and Performance Improvement

By observing the evolution of the feature weights over time as we perform meta-learning, can you justify the improvement in performance?

Yes, the improvement is justified by the model learning **feature sparsity/selection**.

- **Observation:** In the bottom-right plot ("Feature weights"), we see a massive spike in the weights for the first few feature indices (roughly indices 0–10), while the weights for the remaining features (indices 20–500) remain low (near the initialization value of 1.0 or lower).
- **Justification:** The meta-learning process optimizes the feature weights to minimize the loss across different tasks. It learned that the first few features consistently contain the "signal" (the true function $f(x)$), while the other features contain only noise. By assigning high weights to the signal features, the model amplifies the relevant information. This increases the **Signal-to-Noise Ratio (SNR)**, allowing the inner-loop logistic regression to converge to the correct solution even when provided with very few data points.

Specifically, can you explain why some feature weights are being driven towards zero?

Feature weights corresponding to irrelevant dimensions are driven towards zero (or kept low relative to the signal) to **prevent overfitting and reduce variance**.

- **Reasoning:** In this setup, most of the input dimensions are "noise" features that are uncorrelated with the true label z_i .
 1. **Overfitting:** If the model pays attention to these noise features (high weights), the logistic regression in the inner loop will try to "fit" the random noise in the training data, leading to poor generalization on the test set.
 2. **Variance Reduction:** By driving the weights of these noise features toward zero, the meta-learner effectively performs **soft feature selection**. It forces the inner-loop model to ignore the noise. This reduces the variance of the estimator, ensuring that the model's predictions are based only on the features that actually govern the data distribution (D_T).

VAE

Me: Fill in the TODOs in `#vae.py` and `#utils.py`

Gemini Pro:

```
def sample_gaussian(m, v):  
  
    """  
  
    Element-wise application reparameterization trick to sample from  
    Gaussian  
  
    Args:  
  
    m: tensor: (batch, ...): Mean  
  
    v: tensor: (batch, ...): Variance
```

Return:

```
z: tensor: (batch, ...): Samples
```

```
"""
```

```
#####  
#####
```

```
# TODO: Modify/complete the code here
```

```
# Sample z
```

```
#####  
#####
```

```
eps = torch.randn_like(v)
```

```
z = m + eps * torch.sqrt(v)
```

```
#####  
#####
```

```
# End of code modification
```

```
#####  
#####
```

```
return z
```

```
def negative_elbo_bound(self, x):
```

```
"""
```

```
Computes the Evidence Lower Bound, KL and, Reconstruction costs
```

```
Args:
```

```
x: tensor: (batch, dim): Observations
```

```
Returns:
```

```
nelbo: tensor: (): Negative evidence lower bound
```

```
kl: tensor: (): ELBO KL divergence to prior
```

```
rec: tensor: (): ELBO Reconstruction term
```

```
"""
```

```
#####  
#####
```

```
# TODO: Modify/complete the code here
```

```
# Compute negative Evidence Lower Bound and its KL and Rec  
decomposition
```

```
#
```

```
# Note that nelbo = kl + rec
```

```
#
```

```
# Outputs should all be scalar
```

```
#####  
#####
```

```
mean, var = self.enc.encode(x)
```

```
z = ut.sample_gaussian(mean, var)
```

```
logits = self.dec.decode(z)
```

```

# compute metrics

kl = ut.kl_normal(mean, var, self.z_prior[0], self.z_prior[1])

rec = -ut.log_bernoulli_with_logits(x, logits)

nelbo = kl + rec


nelbo, kl, rec = nelbo.mean(), kl.mean(), rec.mean()

#####
#####

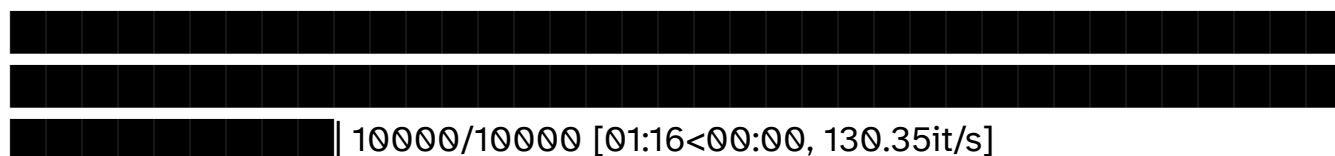
# End of code modification

#####
#####

return nelbo, kl, rec

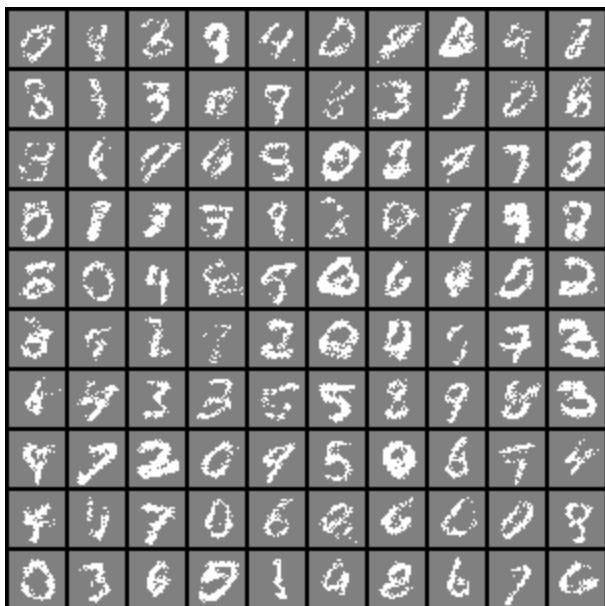
```

100%|



LOG-LIKELIHOOD LOWER BOUNDS ON TEST SUBSET

NELBO: 98.52332305908203. KL: 24.971385955810547. Rec: 73.55194091796875
 Saved visualization to results/generated_vae_10000.png



Me: explain what the code does, what its doing, and how it workst

Gemini:

This code implements a **Variational Autoencoder (VAE)** for the MNIST dataset using PyTorch. It's designed to learn a compressed latent representation of handwritten digits and generate new ones.

Here is a breakdown of how it works:

1. The Core Logic (**vae.py**)

The **VAE** class is the main model. It combines an **Encoder** and a **Decoder** to learn a probability distribution over the data.

- **Encoder (`self.enc`)**: Takes an input image x (flattened 784 pixels) and maps it to a latent distribution $q(z|x)$. Instead of outputting a single vector z , it outputs the **mean (μ)** and **variance (σ^2)** of a Gaussian distribution.
- **Sampling (`ut.sample_gaussian`)**: This is the "Reparameterization Trick". We sample a latent vector z from the distribution defined by μ and σ^2 . This allows gradients to flow back through the sampling step during training.
 - Formula: $z = \mu + \epsilon \cdot \sigma$, where $\epsilon \sim N(0,1)$.
- **Decoder (`self.dec`)**: Takes the sampled latent vector z and reconstructs the original image $\hat{x}$. It outputs **logits**, which are unnormalized probabilities for each pixel (since MNIST pixels are treated as binary values here).

2. The Objective Function (`negative_elbo_bound`)

The model is trained to minimize the **Negative Evidence Lower Bound (NELBO)**, which consists of two terms:

1. **Reconstruction Loss** (`rec`): Measures how well the decoder reconstructs the input `x` from `z`. It uses the negative log-likelihood of a Bernoulli distribution (`ut.log_bernoulli_with_logits`).
2. **KL Divergence** (`kl`): A regularizer that forces the learned latent distribution $q(z|x)$ to be close to a standard normal prior $p(z)=N(0,1)$. This ensures the latent space is smooth and continuous, allowing for valid generation.

Total Loss = KL Divergence + Reconstruction Loss

3. Utility Functions (`utils.py`)

This file contains the mathematical heavy lifting:

- **sample_gaussian**: Implements the reparameterization trick described above.
- **kl_normal**: Computes the analytical KL divergence between two Gaussian distributions.
- **log_bernoulli_with_logits**: Computes the reconstruction loss for binary data.
- **gaussian_parameters**: Splits the encoder's raw output into mean and variance components, ensuring variance is always positive using `softplus`.

4. Neural Networks (`nns.py`)

- **Encoder**: A simple Multi-Layer Perceptron (MLP) with layers `784 -> 300 -> 300 -> 2*z_dim`.
- **Decoder**: An MLP with layers `z_dim -> 300 -> 300 -> 784`.

Summary of Flow

1. **Input**: Image `x`.
2. **Encode**: `x` $\rightarrow$ `mean`, `var`.
3. **Sample**: `z` sampled from $N(\text{mean}, \text{var})$.
4. **Decode**: `z` $\rightarrow$ `logits`.

5. **Loss**: Calculate how different **logits** are

from $\hat{x}$ (Reconstruction) + how different $N(\text{mean}, \text{var})N(\text{mean}, \text{var})$ is
from $N(0,1)N(0,1)$ (KL).